
Práctica 10: filtros

Objetivo: Jugar con filtros de postprocesado

1. Instrucciones

La práctica se realiza sobre un proyecto de Unity en el que se van añadiendo escenas que utilizan renderers distintos configurados para utilizar las *Render features* implementadas en cada apartado.

Se entregará el proyecto completo de Unity, dejando únicamente los directorios necesarios (Assets, Packages y ProjectSettings). En el directorio Assets habrá, además, una carpeta Capturas con imágenes de pantalla.

Por cada apartado de la práctica aparecerá un directorio con todos los ficheros creados en ese apartado. El nombre del directorio comenzará con el número del apartado seguido de su título en *camel case*. Por ejemplo, para el primer apartado habrá un directorio llamado 01ToonShading. Los nombres de los ficheros del directorio comenzarán siempre con el título en *camel case* (en el primer apartado, todos empezarán por ToonShading). Deberán aparecer:

- Un fichero con la escena. Su nombre terminará con el sufijo `_Scene`.
- Un fichero con el *renderer* utilizado por la cámara. Su nombre terminará con el sufijo `_Renderer`.
- Un fichero con el shader implementado para conseguir el efecto.
- Dos ficheros con la *render feature* y *render pass* implementadas en el apartado. Sus nombres terminarán con los sufijos `Feature` y `Pass` y coincidirán con los nombres de las clases que implementan.

Los nombres de las pasadas y las texturas auxiliares que se crean deben coincidir con el nombre del apartado, por ejemplo ToonShading o SimpleBlur.

Igual que en la práctica anterior, las *render feature* *no* tendrán como parámetro el shader/material que hay que utilizar. Éste será cargado desde código a partir de su nombre.

Por cada apartado, además, deben entregarse tres capturas en el directorio Capturas. El nombre de las tres capturas comenzarán con el número de apartado seguido del título

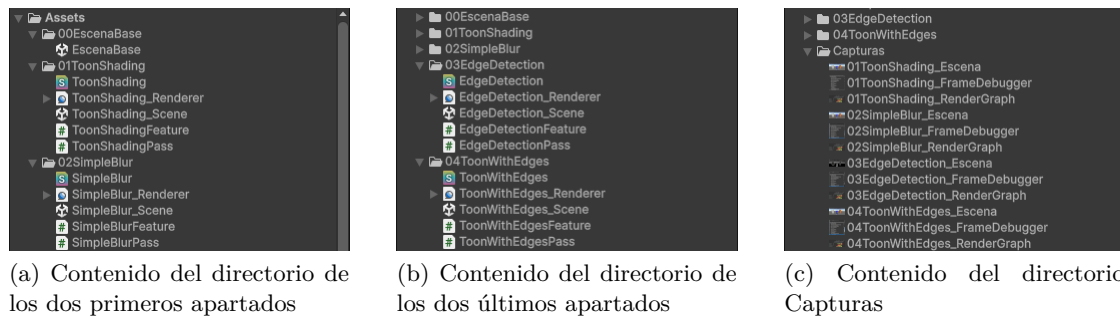


Figura 1: Contenido del directorio Assets de la entrega

(igual que el nombre del directorio) y terminarán con los sufijos `_Escena` (para la captura con la escena del juego), `_FrameDebugger` (para la captura con el frame `FrameDebugger` en donde se vea la etapa añadida) y `_RenderGraph` (para la captura en la que se vea el render graph).

El proyecto hará uso del *render pipeline asset* por defecto creado por Unity al construir el proyecto y se configurará para que tenga disponibles los renderers de cada apartado.

2. Punto de partida

La práctica parte de un proyecto 3D en Unity que utiliza el *Universal Render Pipeline*. Junto con el enunciado se proporciona el proyecto que tiene únicamente la escena base de todos los apartados junto con los recursos necesarios (materiales y textura).

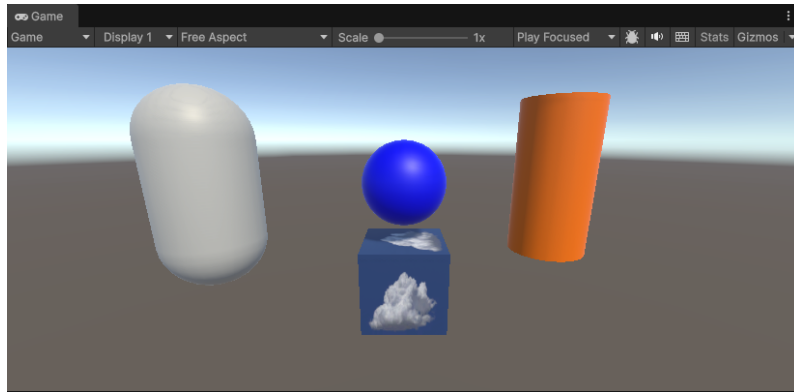
La escena se ha creado a partir de los siguientes materiales, basados en el shader por defecto de URP, Universal Render Pipeline/Lit:

- Azul que utilice el RGB (0, 0, 255).
- Naranja con (255, 120, 0).
- GrisClaro con (200, 200, 200).
- Nube con la textura de una nube que acompaña al enunciado y que está sacada de un ejemplo de renderizado utilizando pbrt-v4 (<https://github.com/mmp/pbrt-v4>).

Coloca en la escena los siguientes objetos:

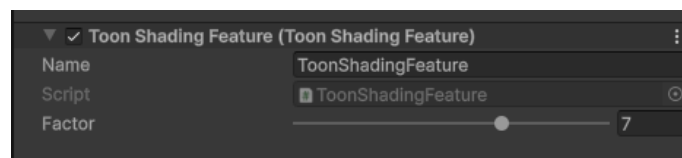
- Cápsula GrisClaro en L: (-2, 0.5, 0).
- Esfera Azul en L: (0, 0.5, 0.5).
- Cilindro Naranja en L: (2, 0.5, 1).
- Cubo con Nube en L: (0, -0.7, 0.5) y R:(0, 90, 0).

La cámara está en L: (0, 1, -3) y R: (15, 0, 0).



3. Toon shading

Una forma fácil de simular el *toon shading* es “discretizando” los colores. Implementa una *Render feature* y su pasada correspondiente que se ejecute tras el postprocesado y que discretice cada componente de color en un número de “escalones” configurable desde la pasada.

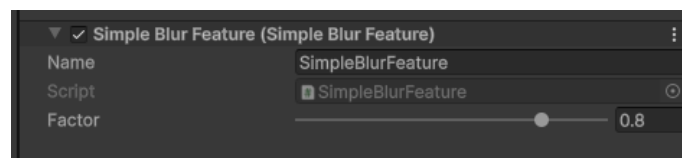


4. Desenfoque simple (*SimpleBlur*)

Implementa una nueva *Renderer feature* y su pasada correspondiente que permita desenfocar mínimamente la escena. Para eso, puedes utilizar el *kernel*:

$$\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} \quad (10.1)$$

La *render feature* debe permitir especificar el grado de uso del desenfoque deseado entre 0 y 1, con 0 indicando que no hay ningún tipo de desenfoque y 1 con el efecto aplicado por completo.



Ten en cuenta que si el factor configurado es 0, la etapa no deberá ejecutarse.

5. Detección de aristas (*EdgeDetection*)

Implementa una nueva *Renderer feature* y su pasada correspondiente que detecte las aristas de la escena. Para eso

La *feature* permitirá configurar el color de la silueta que se puede combinar la detección de las aristas horizontales con el kernel

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} \quad (10.2)$$

con las aristas verticales utilizando el kernel análogo

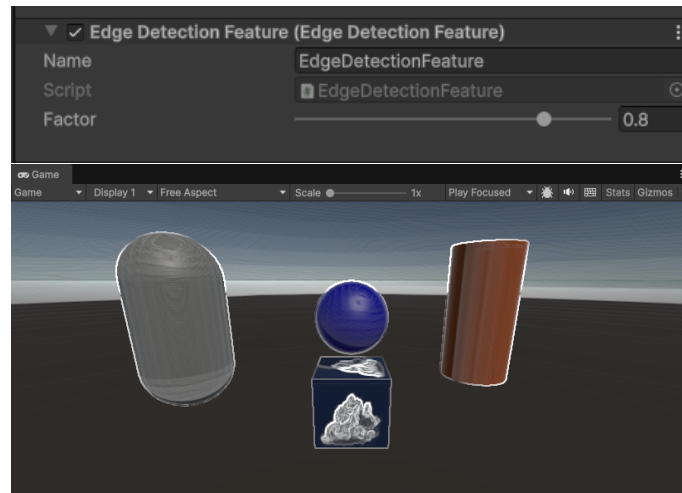
$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad (10.3)$$

Ambos kernels se aplican por separado utilizando la intensidad de cada píxel (convirtiendo el color a escala de grises). La magnitud que indica “cómo de arista” es el pixel la da la fórmula

```
float mag = length(float2(h, v));
```

con *h* y *v* el resultado de los dos *kernel*.

Utiliza, igual que en la práctica anterior, un factor de mezcla en donde 0 significa pintar la escena original y el 1 el color que representa la intensidad de la arista (`float4(mag, mag, mag, 1.0)`).

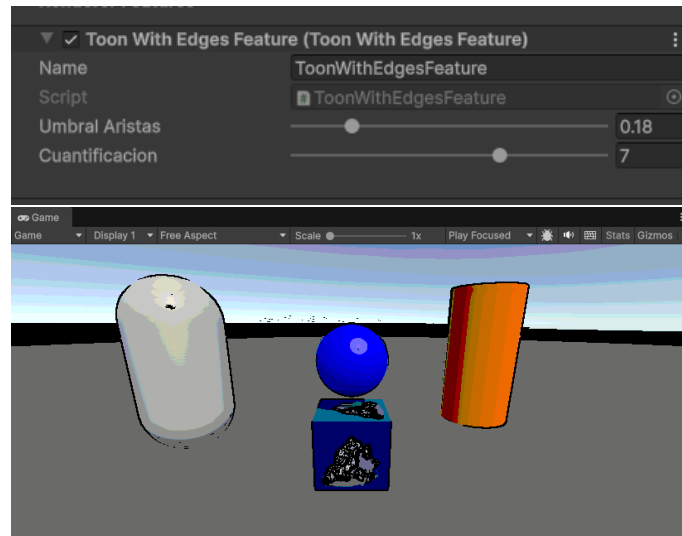


6. Toon shading con aristas (*Toon With Edges*)

Combina los dos efectos anteriores en una nueva *render feature* que calcule el “nivel de arista” de cada pixel; en caso de superar un cierto umbral configurable, pintará ese color de negro (lo considerará arista) y en caso contrario le aplicará el efecto de *toon shading*.

7. Gaussian Blur

Implementa un efecto de desenfoque mejorado en el que



8. Entrega de la práctica

La práctica debe entregarse utilizando el mecanismo de entrega del campus virtual, no más tarde de la fecha y hora indicada en la cabecera de la práctica.

Solo uno de los dos miembros del grupo debe hacer la entrega, subiendo al campus un fichero llamado `GrupoNN.zip` donde NN representa el número de grupo con dos dígitos.

El fichero debe tener exclusivamente:

- El proyecto de Unity con los directorios mínimos necesarios (`Assets`, `Packages` y `ProjectSettings`). El contenido del directorio `Assets` debe ser el que se deduce de las instrucciones anteriores y que puede verse en la figura 1.
- Un fichero `alumnos.txt` donde se indicará el nombre de los componentes del grupo.